

20. Deux touches, deux effets

Un `enum` donne des noms à des nombres, et `switch` les range côte à côte.

1. Ce qu'il faut comprendre

`TITRE` vaut 0, `JEU` vaut 1, `FIN` vaut 2 — mais **on ne l'écrit jamais**, et c'est tout l'intérêt : ajouter une scène au milieu ne renumérote rien à la main.

`Scene` devient un **type d'un octet**. `Scene ou = TITRE;` se relit mieux que `uint8_t ou = 0;`, et dit ce que la variable a le droit de contenir.

C'est la forme que prend presque tout jeu : quelques scènes, et un `switch` qui dit laquelle est en cours.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **617 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         111 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
LireManette    37 o ← bouton() - lit les huit boutons
Tuiles         208 o ← CopierTuiles - 13 lettres sur 43
"APPUIE START" 12 o ← texte() - un texte gravé
"              " 12 o ← texte() - un texte gravé
"ON JOUE      " 12 o ← texte() - un texte gravé
"C EST FINI   " 12 o ← texte() - un texte gravé
```

Lettres gravées : ACEFIJNOPRSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 4    44 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 2    10 o sur place, dans main() + LireManette (37 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : trois scènes. Le TITRE (START pour jouer), le JEU
// (SELECT pour finir), la FIN.
//
// enum Scene ... : invente un type « Scene » dont les valeurs ont des noms :
//   TITRE (vaut 0), JEU (vaut 1), FIN (vaut 2).
// switch (ou) ... case X: ... break; : saute directement au cas qui correspond
//   à la valeur de ou ; « break » sort du switch.

#include <texte>    // écrit un texte à l'écran
#include <bouton>   // lit un bouton de la manette
```

2. DES CONSTANTES NOMMÉES — SCENE

```
enum Scene { TITRE, JEU, FIN };    // Les trois scènes possibles.
```

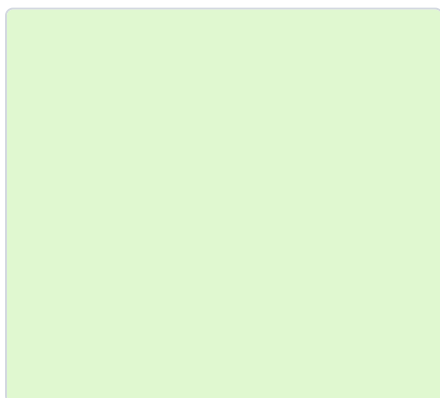
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                      // Le jeu commence ici.
    Scene ou = TITRE;             // La scène actuelle : on commence au titre.

    while (true) {                // Répète pour toujours :
        switch (ou) {             // Selon la scène :
            case TITRE:           // --- au titre :
                texte(4, 8, "APPUIE START");
                if (bouton(START)) { texte(4, 8, "          "); ou = JEU; }
                                //   START : on efface (12 espaces), et on passe au JEU.
                break;
            case JEU:             // --- pendant le jeu :
                texte(4, 8, "ON JOUE  ");
                if (bouton(SELECT)) ou = FIN; //   SELECT : on passe à la FIN.
                break;
            case FIN:             // --- à la fin :
                texte(4, 8, "C EST FINI "); //   on reste ici pour toujours.
                break;
        }
        image();                  // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

START passe au jeu, SELECT y met fin.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ on démarre sur le titre
- ✓ START passe au jeu
- ✓ SELECT y met fin
- ✓ et l'on y reste

Ensuite : **20.1. Chapitre 4 — Les conditions : choisir : ce qui vient** — chapitre 4 : Les conditions : choisir. · [gameboy3](#) — écrire une cartouche Game Boy en C++